

Tabelas Hash

Disciplina: Estrutura de Dados II

Curso: Engenharia da Computação - 4º Período

Prof. Ana Figueiredo

Instituto Federal Fluminense

May 26, 2025

Sumário

- 1 Introdução
- 2 Função Hash
- 3 Colisão
- 4 Tamanho da Tabela
- 5 Exemplos de Funções Hash
- 6 Tratamento de Colisão

O que é uma Tabela Hash?

Tabela Hash:

- Estrutura de dados que associa **chaves** a **valores** de forma eficiente.
- Utiliza uma **função hash** para transformar a chave em um índice de uma tabela.
- Busca, inserção e remoção em tempo médio $\mathcal{O}(1)$.

Exemplos práticos:

- Tabelas de símbolos em compiladores.
- Indexação em bancos de dados.
- Sistemas de autenticação e cache.

Vantagens e Desvantagens

Vantagens:

- Eficiência: operações rápidas.
- Simplicidade conceitual.

Desvantagens:

- Problemas com **colisões**.
- Dependência de uma **boa função hash**.
- Dificuldade para crescer dinamicamente.

Função Hash

Definição:

- Função que mapeia uma chave de entrada para um índice na tabela.

Exemplo usado no nosso código:

$$h(k) = k \mod T$$

onde k é a chave e T o tamanho da tabela.

Inserção no código:

```
5 int hashFunc(int chave) {  
6     return chave % TAM;  
7 }
```

Imagem: função hash simples baseada no operador módulo.

Hash Perfeito, Imperfeito e Universal

Hash Perfeito:

- Sem colisões.
- Impraticável na maioria dos casos.

Hash Imperfeito:

- Pode gerar colisões.
- Mais realista e utilizado.

Hashing Universal:

- Conjunto de funções hash.
- Seleccionada aleatoriamente para minimizar colisões.
- Importante em cenários adversos.

O que é Colisão?

- Ocorre quando duas chaves distintas são mapeadas para o mesmo índice.
- Fenômeno inevitável quando o domínio das chaves é maior que o tamanho da tabela.

Causas:

- Função hash inadequada.
- Tabela muito pequena.

Problemas:

- Perda de eficiência.
- Erros de sobrescrição se não tratadas.

Exemplo:

- Chaves: 10 e 20. Tamanho da tabela: 10.
- Ambas serão mapeadas para o índice 0.

Nosso código ainda não trata colisões!

Importância do Tamanho da Tabela

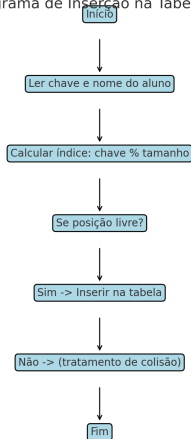
- Influencia diretamente na quantidade de colisões.
- **Tabela pequena**: muitas colisões.
- **Tabela grande**: desperdício de memória.

Boas práticas:

- Usar tamanho **primo** para melhorar dispersão.
- Ajustar conforme o **fator de carga**: $\alpha = \frac{n}{T}$

Fluxo de Inserção

Fluxograma de Inserção na Tabela Hash



```
26 void inserir(TabelaHash* th, int matricula, string& nome) {  
27     int indice = hashFunc(matricula);  
28     if (th->tabela[indice] == nullptr) {  
29         th->tabela[indice] = new Aluno(matricula, nome, true);  
30         cout << "Aluno inserido na posicao " << indice << endl;  
31     } else {  
32         cout << "Colisao! Posicao " << indice << " ja ocupada." << endl;  
33     }  
34 }
```

Funções Hash Numéricas

```
12 int chaveMultiplicacao(int chave, int TABLE_SIZE) {  
13     float A = 0.6180339887; // constante:  $0 < A < 1$   
14     float val = chave * A;  
15     val = val - (int) val;  
16     return (int) (TABLE_SIZE * val);  
17 }
```

Imagem: inserção usando função hash por multiplicação.

Hashing com Strings

Método comum:

$$h(s) = \sum_{i=0}^{n-1} s_i \cdot p^i \mod m$$

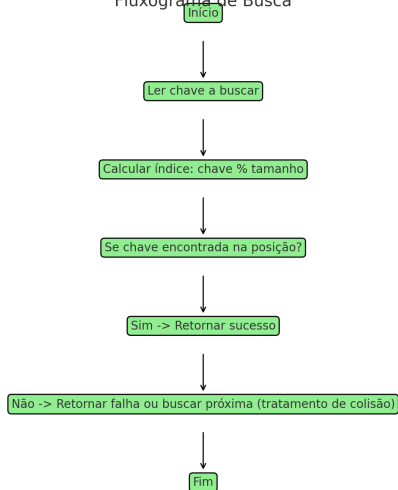
- s_i : código ASCII do caractere.
- p : número primo, ex.: 31.

Exemplo:

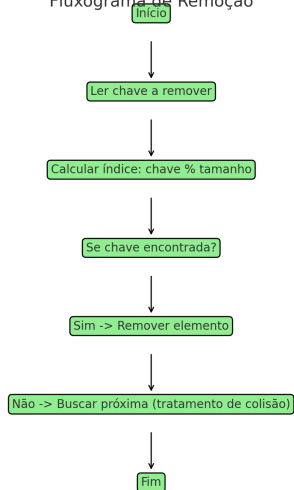
```
function HASHSTRING(s, m)
   $h \leftarrow 0$ 
  for all caractere  $c$  em  $s$  do
     $h \leftarrow (h \times 31 + c) \mod m$ 
  end for
  return  $h$ 
end function
```

Fluxograma de Busca e Remoção

Fluxograma de Busca



Fluxograma de Remoção



Duas abordagens principais:

- 1 **Endereçamento Aberto**
- 2 **Encadeamento Separado**

Nosso código atual não trata colisões!
(atividade proposta ao final).

Endereçamento Aberto

Como funciona:

- Em caso de colisão, busca-se a próxima posição livre.

Técnicas:

- **Sondagem Linear:** próxima posição sequencial.
- **Sondagem Quadrática:** salto crescente.
- **Duplo Hashing:** nova função hash determina o salto.

Pseudocódigo:

```
function INSERT( $k$ )  
   $i \leftarrow 0$   
  repeat  
     $pos \leftarrow (h(k) + i) \bmod m$   
     $i \leftarrow i + 1$   
  until posição livre  
  insere  $k$  na posição  $pos$   
end function
```

Encadeamento Separado

Como funciona:

- Cada posição armazena uma lista de elementos.
- Elementos que colidem são encadeados.

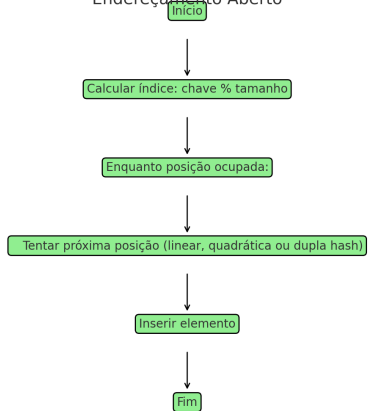
Exemplo:

```
function INSERT( $k$ )  
     $pos \leftarrow h(k)$   
    Insere  $k$  na lista encadeada em  $pos$   
end function
```

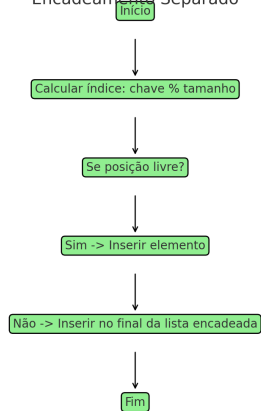
Vantagem: evita necessidade de encontrar nova posição. **Desvantagem:** custo adicional de gerenciamento das listas.

Tratamento colisão

Endereçamento Aberto



Encadeamento Separado



Atividade Proposta

Durante os slides, uma versão de tabela hash bem simples foi apresentada, o código será fornecido.

- Implemente as demais funções de hash:
 - Multiplicação.
 - Hashing com Strings.
- Implemente as duas formas de tratamento de colisão:
 - Encadeamento separado.
 - Endereçamento aberto.

Desafio extra: compare o desempenho entre as abordagens.

Tabelas Hash

Disciplina: Estrutura de Dados II

Curso: Engenharia da Computação - 4º Período

Prof. Ana Figueiredo

Instituto Federal Fluminense

May 26, 2025